

Enhancing Permissioned Blockchains with Controlled Data Authorization

Qichang Liu¹, Xufeng Zhang², Sisi Duan¹, and Haibin Zhang³(✉)

¹ Tsinghua University, Beijing, China

liuqc21@mails.tsinghua.edu.cn, duansisi@tsinghua.edu.cn

² Beijing Institute of Technology, Beijing, China

xufengzhang@bit.edu.cn

³ Yangtze Delta Region Institute of Tsinghua University, Zhejiang, China

bchainzhang@aliyun.com

Abstract. Blockchains provide data with availability and integrity. Many applications, however, also require strong confidentiality, a goal that conventional blockchains fail to achieve. This work enhances permissioned blockchains based on Byzantine fault tolerance (BFT) with the ability to manage confidential data in a controlled manner. For this goal, we formally define threshold encryption with controlled authorization (ECA). In an ECA scheme, writers can send transaction messages to readers via servers (e.g., blockchain servers). There are many challenges for ECA — decoupling writers and readers, message confidentiality, reliability of the system, and the tolerance of malicious behaviors. We define the syntax for the ECA system and formalize three security notions for ECA, namely *token verifiability*, *t-robustness*, and *t-collusion resistant CPA security*. We propose an ECA scheme and provide formal proofs of these notions for our proposed ECA scheme. Our ECA scheme decouples writers and readers with the help of servers. The *token verifiability* guarantees all the servers obtain the right tokens from writers. The *t-robustness* makes sure our ECA scheme runs well even if t malicious servers implement active attacks. The *t-collusion resistant CPA security* guarantees message privacy from servers, even if t servers collude with each other to implement active attacks. Finally, we offer an efficient ECA construction satisfying all these security notions.

Keywords: Threshold encryption · Controlled authorization · Permissioned blockchain · Confidentiality · Fine-grained access control.

1 Introduction

Blockchains allow the exchange of digital data and assets among users in a transparent and decentralized manner. They can be generally divided into permissionless blockchains (e.g. Bitcoin, Ethereum), where the servers can join and leave the system in a permissionless manner, and permissioned blockchains (e.g. Hyperledger Fabric, Diem), where the servers are coordinated in a controlled and authenticated manner. Blockchains, no matter whether permissionless or

permissioned, can typically enforce data consistency and availability only, but not data confidentiality. Therefore, it is challenging to use blockchains for applications that need to protect sensitive user data (e.g., medical records, personal bank accounts, insurance information). This paper focuses mainly on enabling fine-grained confidentiality for permissioned blockchains.

TWO POPULAR CONFIDENTIALITY GOALS IN PERMISSIONED BLOCKCHAINS. There are two important confidentiality goals in permissioned blockchains.

One such goal is causality preservation [27, 7, 14]. In replicated services, when a correct client issues a transaction, some faulty replicas may build a new transaction that can be executed before the original request of the client. Consider a stock trading service, where a client issues a transaction to buy some stock, and a faulty replica can collude with another faulty client to send a derived transaction for the same stock. The new transaction may be executed before the transaction of the correct client, adjusting the demand for the stock and raising the price to the correct client.

A second notion is confidentiality with access control [13, 22]. In this setting, the transactions should be encrypted by the owners of the transactions and stored in the blockchain service. The owners of transactions can specify the access control policy of the specific transactions, while blockchain servers collectively enforce the access control policy and engage with the readers (who satisfy the access control policy) such that readers obtain the underlying transactions.

It is important to note that one may use threshold encryption [28, 2, 6] in permissioned blockchains for both two confidentiality goals. In this approach, we associate a public-key threshold encryption scheme with permissioned blockchains. The blockchain servers jointly generate a single public key and each server holds its own decryption key. The clients encrypt their transactions with the public key of the threshold encryption and send the ciphertexts and corresponding access control lists¹. Servers run the underlying consensus protocol to reach an agreed total order on the ciphertexts together with corresponding access control lists that needs to be enforced by the servers. Upon receiving a request for some encrypted transactions, if the request is from one who is in the access control list, each server can use its corresponding decryption key to generate its decryption share and send it to the client. Then the client can combine the decryption shares to obtain the underlying plaintext.

Meanwhile, by using threshold encryption, the malicious blockchain servers cannot see the underlying transactions before them are committed and finalized. Therefore, the adversary cannot perform causality-related attacks and front-running attacks, so the blockchain service naturally achieves causality and fairness preservation.

In this paradigm, unless more than a threshold (typically, $1/3$) of blockchain servers collude, the underlying transactions will be well protected. However, ever since the threshold encryption ciphertext is uploaded to the blockchain service, the service immediately gets the ability to decrypt the ciphertext. If a

¹ Strictly speaking, one should pick a labeled threshold encryption scheme where the label of the encryption now should be the access control list [28]

dedicated adversary corrupts servers one by one and eventually controls more than a threshold of servers, it immediately learns all the underlying transactions. Doing so is problematic in practice because when the service acts maliciously at some point, the underlying plaintext can be directly obtained for the service, disabling all the control of the transaction owners.

THRESHOLD ENCRYPTION WITH CONTROLLED AUTHORIZATION. To solve the above-mentioned issue, we need a controlled confidentiality solution for permissioned blockchain services, where the clients can decide when to delegate the decryption ability to the servers. Namely, after clients write the ciphertexts and before they delegate the decryption ability, the blockchain servers cannot decrypt the associated ciphertexts even though more than a threshold of blockchain servers collude. The clients can choose to delegate the decryption ability at any point (may it be triggered by an external event, or simply due to their own choices). Note that the decryption ability can be enforced in a per-transaction manner, thereby offering fine-grained control over access control and stronger security against dedicated attacks on blockchain servers. (Note that another attempting approach is to refrain from sending (encrypted) transactions to the blockchain servers until the point when the readers request them. While in this way the senders may seem to be able to control the data authorization, there is an obvious drawback. Indeed, to ensure transaction fairness and defend against front-running attacks, the senders should send their ciphertexts once they are formed; otherwise, the system would lose fairness.)

To this goal, we define threshold encryption with controlled authorization, or simply ECA, illustrated by Fig. 1. As threshold encryption, ECA should be compatible with blockchains or consensus protocols underlying blockchains. More specifically, a writer $\mathcal{W}$ can encrypt a message msg and send the ciphertext ct along with some access control list $\text{acl}_{\mathcal{W}}$ to the blockchain servers. Then the servers run the consensus protocol underlying blockchains to agree on the ciphertext ct and $\text{acl}_{\mathcal{W}}$. Doing so will ensure the ciphertext ct is committed while preventing causality-related attacks. Later, the writer can authorize the blockchain servers via tokens to convert the ciphertext ct for some specific readers in the list $\text{acl}_{\mathcal{W}}$. Note the Certify process is needed to defend against malicious writers who may send inconsistent tokens.

OUR CONTRIBUTIONS. In this paper, we make the following contributions:

- We define the primitive of threshold encryption with controlled authorization (ECA), endowing with it several security notions capturing the needs of blockchain services requiring fine-grained access control and causality preservation.
- We provide a novel ECA scheme satisfying the security notions we define. We develop formal security proofs for our ECA scheme with a novel security reduction to the square Decisional Diffie-Hellman (DDH) assumption [10].

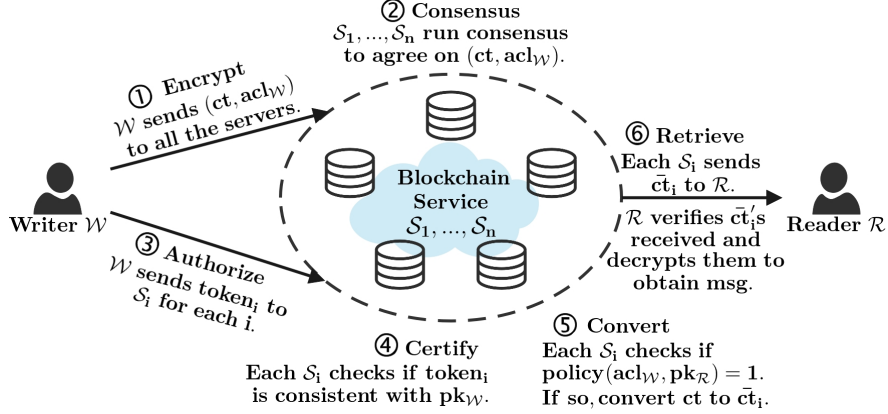


Fig. 1. ECA in the blockchain setting. The Encrypt and Authorize requests should be in the form of blockchain write transactions, the consensus process is executed by the blockchain ordering service (that is used to ensure transactions are committed atomically), and the Retrieve process resembles the blockchain read service. Besides, the Certify and Convert processes can be realized as state update operations in Byzantine fault-tolerant state machine replication or smart contract operations. Alternatively, the framework could be extended to the setting where the readers can express their interests to blockchain servers, and the writers authorize the decryption upon receiving the requests forwarded from the servers. In either case, the framework decouples writers from readers, providing scalability and flexibility.

2 Related Work

PERMISSIONED BLOCKCHAINS VS. PERMISSIONLESS BLOCKCHAINS. Blockchains are divided into permissionless blockchains (e.g. Bitcoin [25], Ethereum [30]) and permissioned blockchains (e.g. Hyperledger Fabric [1]). Byzantine fault tolerance (BFT) has long been understood as the standard model for permissioned blockchains. BFT is also widely and increasingly used in various permissionless blockchains (e.g., the latest version of Ethereum).

OTHER CONFIDENTIALITY NOTIONS IN PERMISSIONED BLOCKCHAINS. Besides causality preservation and confidentiality with access control, several other confidentiality notions make sense in the permissioned blockchain setting. However, these notions are out of the scope of the paper.

For instance, BFT protocols can be integrated with multi-party computation (MPC) protocols. Such an integration allows processing client-initiated transactions in a privacy-preserving manner. In fact, MPC protocols achieving liveness (guaranteed output delivery) have to use some underlying consensus protocols [3, 4, 24, 15]. In the blockchain context, blockchains may be equipped with private smart contracts to process transactions. Note that one may also use trusted execution environments (TEEs) for the same purpose, but TEEs require a strong trust assumption.

One alternative approach is to separate agreement from execution [31, 16] and use reliable filters. However, this approach requires an architecture that uses more replicas.

ECA vs. THRESHOLD ENCRYPTION. ECA is stronger than threshold encryption [28] in the sense that ECA allows more fine-grained control over the ciphertexts, i.e., the decryption in ECA is controlled by the encrypter of the transactions. In practice, the unavailability of the encrypter does not necessarily impact the liveness of the system, because it is the encrypter that decides when to delegate the decryption ability; indeed, such an ability can be simultaneously transferred together with the ciphertexts to some third parties.

ECA vs. PROXY RE-ENCRYPTION. ECA is different from proxy re-encryption (PRE) [5, 33] in two aspects. First, PRE requires a centralized proxy, while ECA is inherently distributed. Second, ECA in fact has a process that fully delegates its decryption ability to some parties, while PRE never delegates the full decryption power to any proxy.

VSS. Verifiable secret sharing [11] allows a dealer to distribute secret shares in a verifiable manner and ensures consistency even in the presence of a faulty dealer. In practice, VSS works in synchronous settings and asynchronous settings [17, 8] (relying on no timing assumptions). The token verifiability notion captures a similar notion ensuring consistency for secret shares and is currently modeled in synchronous environments. Meanwhile, our construction can be easily adapted to achieve asynchronous token verifiability if using techniques of asynchronous VSS (AVSS).

3 Preliminaries

Notations. Let $\kappa \in \mathbb{N}$ denote the security parameter throughout the paper, and all algorithms take 1^κ as an implicit input. Let $\emptyset$ denote the empty set. If x is defined by y or the value of y is assigned to x , we write $x := y$. For $i \in \mathbb{N}$, define $[i] := \{1, 2, \dots, i\}$. For $i, j \in \mathbb{Z}$ with $i < j$, define $[i, j] := \{i, i+1, \dots, j\}$. For a set $\mathcal{X}$, denote by $x \leftarrow_s \mathcal{X}$ the procedure of sampling x from $\mathcal{X}$ uniformly at random. If X is distribution, $x \leftarrow_s X$ means that x is sampled according to X . All our algorithms are probabilistic unless stated otherwise. We use $y \leftarrow \mathcal{A}(x)$ to define the random variable y obtained by executing algorithm $\mathcal{A}$ on input x . We use $y \in \mathcal{A}(x)$ to indicate that y lies in the support of $\mathcal{A}(x)$. We also use $y \leftarrow \mathcal{A}(x; r)$ to make explicit the random coins r used in the probabilistic computation. “PPT” abbreviates probabilistic polynomial-time. Denote by poly some polynomial function and negl some negligible function. By $\Pr_i[\cdot]$ we denote the probability of a particular event occurring in game G_i .

Two distributions X and Y are computationally indistinguishable, if $|\Pr[\mathcal{D}(X) \Rightarrow 1] - \Pr[\mathcal{D}(Y) \Rightarrow 1]| \leq \text{negl}(\kappa)$ for all PPT distinguisher $\mathcal{D}$. We denote it by $X \approx_c Y$.

Definition 1 (SE and its Semantic Security). $\text{SE} = (\text{SE.Enc}, \text{SE.Dec})$ is an symmetric encryption scheme where $\text{ct} \leftarrow \text{SE.Enc}(k, m)$ and $m \leftarrow \text{SE.Dec}(k, \text{ct})$

are encryption and decryption algorithms. The correctness of SE requires that $\text{SE.Dec}(k, \text{SE.Enc}(k, m)) = m$. The semantic security of SE requires that for any two plaintexts m_0, m_1 of equal length, it holds that

$$\text{SE.Enc}(k, m_0) \approx_c \text{SE.Enc}(k, m_1),$$

where k is uniformly distributed over the key space.

Definition 2 (q -DDH Assumption). Let $\mathbb{G}$ be a cyclic group of prime order p with generator g . The q -DDH Assumption holds over $\mathbb{G}$ if

$$(g, g^\alpha, g^{\alpha^2}, \dots, g^{\alpha^q}, g^r, g^{\alpha r}) \approx_c (g, g^\alpha, g^{\alpha^2}, \dots, g^{\alpha^q}, g^r, g^c),$$

where $\alpha, r, c \leftarrow_{\$} \mathbb{Z}_p$.

If $q=1$, the q -DDH assumption is just the traditional DDH assumption. If $q=2$, this assumption is renamed as the *square DDH* assumption. In [10], the hardness of q -DDH problem was justified in the generic group model.

3.1 Lagrange Interpolations

We review the technique of Lagrange Interpolations over $\mathbb{Z}_p$ with p a prime. Recall that $\mathbb{Z}_p$ is isomorphic to a group $\mathbb{G}$ of order p , so Lagrange Interpolations can also be implemented over $\mathbb{G}$.

Lemma 1. Let p be a prime and $n \in \mathbb{Z}^+$ such that $n < p$. Let $r_j \in \mathbb{Z}_p$ with $j \in [0, t]$ and define $r(x) = r_0 + r_1x + \dots + r_tx^t$. If $\theta_i = r(i)$ for $i \in [n]$, then we have the following facts.

- (1) For any $t+1$ distinct value $i_1, \dots, i_{t+1} \in [n]$, the coefficients of polynomial $r(x)$ can be efficiently constructed from $(i_1, \theta_{i_1}), \dots, (i_{t+1}, \theta_{i_{t+1}})$ with Lagrange Interpolation:

$$r(x) = r_0 + r_1x + \dots + r_tx^t = \sum_{j=1}^{t+1} \theta_{i_j} \cdot \prod_{k=1, k \neq j}^{t+1} \frac{x - i_k}{i_j - i_k}. \quad (1)$$

- (1)' For any cyclic group $\mathbb{G}$ of order p with generator g , the isomorphism from $\mathbb{Z}_p$ to $\mathbb{G}$ makes sure that the group elements $(g^{r_0}, g^{r_1}, \dots, g^{r_t})$ can be efficiently constructed from $(i_1, g^{\theta_{i_1}}), \dots, (i_{t+1}, g^{\theta_{i_{t+1}}})$.
- (2) The Lagrange coefficients are defined by $\lambda_{i_j \ell} := \prod_{k=1, k \neq j}^{t+1} \frac{\ell - i_k}{i_j - i_k}$ for $k \in [t+1], \ell \in [0, n]$. Then given any $t+1$ distinct value $i_1, \dots, i_{t+1} \in [n]$, $(\theta_1, \dots, \theta_n)$ can be constructed from $(i_1, \theta_{i_1}), \dots, (i_{t+1}, \theta_{i_{t+1}})$ by

$$\theta_\ell = \sum_{j=1}^{t+1} (\theta_{i_j} \cdot \lambda_{i_j \ell}) \quad \text{for } \ell \in [n]. \quad (2)$$

- (2)' For any cyclic group $\mathbb{G}$ of order p with generator g , the isomorphism from $\mathbb{Z}_p$ to $\mathbb{G}$ makes sure that the group elements $(g^{\theta_1}, g^{\theta_2}, \dots, g^{\theta_n})$ can be efficiently constructed from $(i_1, g^{\theta_{i_1}}), \dots, (i_{t+1}, g^{\theta_{i_{t+1}}})$.

- (3) The distribution of $(r_0, r_1, \dots, r_t, \theta_1, \dots, \theta_n)$, where $r_i \leftarrow_{\$} \mathbb{Z}_p$ for $i \in [t]$ and $\theta_j = r(j)$ for $j \in [n]$, is identical to that of $(r'_0, r'_1, \dots, r'_t, \theta'_1, \dots, \theta'_n)$, where $\theta'_{i_1}, \dots, \theta'_{i_{t+1}} \leftarrow_{\$} \mathbb{Z}_p$, $(\theta'_1, \dots, \theta'_n)$ are computed by (2) and $(r_0, r_1, \dots, r_t)$ is computed by (1).

3.2 Non-Interactive Zero-Knowledge Proof of Equality of Discrete Logarithms.

Let $(h, \bar{h} = h^\theta, g, \bar{g} = g^\theta) \in \mathbb{G}^4$ be a DDH tuple. There is a Sigma protocol providing proof of equality for discrete logarithms $\log_h \bar{h} = \log_g \bar{g}$ [9]. With the standard Fiat-Shamir heuristic, the Sigma protocol can be converted into a non-interactive zero-knowledge proof scheme $\text{NIZK}_{\text{eq}} = (\text{ZKProof}, \text{ZKVerify}, \text{ZKSim})$ for the NP language $\mathcal{L}_{\text{ddh}} = \{(h, \bar{h}, g, \bar{g}) \in \mathbb{G}^4 \mid \log_h \bar{h} = \log_g \bar{g}\}$. Let H be a hash function which is idealized as a random oracle.

- $\pi \leftarrow \text{ZKProof}((h, \bar{h}, g, \bar{g}), \theta)$. The proving algorithm takes as input the DDH tuple $(h, \bar{h}, g, \bar{g})$ and the witness $\theta = \log_h \bar{h} = \log_g \bar{g}$, and it computes a proof π as follows. Sample $s \leftarrow_{\$} \mathbb{Z}_p$ and compute $\hat{h} := h^s$, $\hat{g} := g^s$. Compute $e := H(h, \bar{h}, g, \bar{g}, \hat{h}, \hat{g})$ and $f := s + \theta \cdot e \pmod p$. The proof is $\pi = (\hat{h}, \hat{g}, f)$.
- $0/1 \leftarrow \text{ZKVerify}((h, \bar{h}, g, \bar{g}), \pi)$. The verification algorithm takes as input the instance $(h, \bar{h}, g, \bar{g})$ and a proof π . If $(h^f = \hat{h}\bar{h}^e) \wedge (g^f = \hat{g}\bar{g}^e)$, then it outputs 1 indicating acceptance of the instance. Otherwise, it outputs 0 indicating rejection of the instance.
- $\tilde{\pi} \leftarrow \text{ZKSim}((h, \bar{h}, g, \bar{g}) \in \mathcal{L}_{\text{ddh}})$. The simulation algorithm takes as input the instance $(h, \bar{h}, g, \bar{g})$ and outputs a simulated proof $\tilde{\pi} = (\hat{h}, \hat{g}, f)$ as follows. Sample $s, f \leftarrow_{\$} \mathbb{Z}_p$ and compute $\hat{h} := h^s$, $\hat{g} := g^s$ and $e := (f - s)/\theta \pmod p$. Define $H(h, \bar{h}, g, \bar{g}, \hat{h}, \hat{g}) := e$.

The completeness of NIZK_{eq} is obvious, since if $(h, \bar{h}, g, \bar{g}) \in \mathcal{L}_{\text{ddh}}$, and π is the output of ZKProof , then ZKVerify always outputs 1.

According to [19], the soundness of NIZK_{eq} holds in the random oracle model. For any $(h, \bar{h}, g, \bar{g}) \notin \mathcal{L}_{\text{ddh}}$, any $\pi = (\hat{h}, \hat{g}, f)$ satisfies $(h^f = \hat{h}\bar{h}^e) \wedge (g^f = \hat{g}\bar{g}^e)$ with probability at most $(\ell + 1)/p = \text{negl}(\kappa)$, where ℓ is the number of hash queries. Meanwhile, in the random oracle model, if $(h, \bar{h}, g, \bar{g}) \in \mathcal{L}_{\text{ddh}}$ the distribution of simulated proof $\tilde{\pi}$ from ZKSim is identical to the real proof π generated from ZKProof , except with negligible probability. Therefore, NIZK_{eq} also has the property of zero-knowledge.

4 The Framework of Threshold Encryption with Controlled Authorization

In threshold encryption with controlled authorization (ECA), there are three parties, namely writers $\mathcal{W}$, readers $\mathcal{R}$, and servers $\mathcal{S}$. We assume there are n servers $\mathcal{S}_1, \dots, \mathcal{S}_n$. Now we present a framework for threshold encryption with controlled authorization which can be captured by the following eight algorithms (Setup, KeyGen1, KeyGen2, Encrypt, Authorize, Certify, Convert, Retrieve).

Definition 3 (ECA). A threshold encryption with controlled authorization ECA with n servers can be captured by eight polynomial-time algorithms, namely $\text{ECA} = (\text{Setup}, \text{KeyGen1}, \text{KeyGen2}, \text{Encrypt}, \text{Authorize}, \text{Certify}, \text{Convert}, \text{Retrieve})$.

- $\text{pp} \leftarrow \text{Setup}$. The Setup algorithm outputs a public parameter pp , which serves as an implicit input to the remaining algorithms of ECA.
- $(pk_{\mathcal{W}}, sk_{\mathcal{W}}) \leftarrow \text{KeyGen1}$. The KeyGen1 algorithm takes pp as input and outputs a public key and secret key pair $(pk_{\mathcal{W}}, sk_{\mathcal{W}})$ for a writer $\mathcal{W}$.
- $(pk_{\mathcal{R}}, sk_{\mathcal{R}}) \leftarrow \text{KeyGen2}$. The KeyGen2 algorithm takes pp as input and outputs a public key and secret key pair $(pk_{\mathcal{R}}, sk_{\mathcal{R}})$ for a reader $\mathcal{R}$.
- $(\text{ct}, \text{acl}_{\mathcal{W}}) \leftarrow \text{Encrypt}(pk_{\mathcal{W}}, \text{msg}, \text{acl}_{\mathcal{W}})$. The Encrypt algorithm takes as input the writer's public key $pk_{\mathcal{W}}$, a message $\text{msg} \in \mathcal{M}$ and its associated access control list $\text{acl}_{\mathcal{W}}$, and outputs a ciphertext ct and the access control list $\text{acl}_{\mathcal{W}}$.
- $(\text{token}_1, \dots, \text{token}_n) \leftarrow \text{Authorize}(sk_{\mathcal{W}})$. The Authorize algorithm takes the writer's secret key $sk_{\mathcal{W}}$ as input and outputs n tokens.
- $0/1 \leftarrow \text{Certify}(pk_{\mathcal{W}}, i, \text{token}_i)$. The Certify algorithm takes as input the writer's public key $pk_{\mathcal{W}}$, the server's index i and its token token_i , and it outputs a deciding bit indicating if token_i is valid.
- $\bar{\text{ct}}_i \leftarrow \text{Convert}(\text{token}_i, (pk_{\mathcal{W}}, \text{ct}, \text{acl}_{\mathcal{W}}), pk_{\mathcal{R}}, \text{policy})$. The Convert algorithm takes as input server $\mathcal{S}_i$'s token token_i , the writer's public key $pk_{\mathcal{W}}$ and a ciphertext ct with its access control list $\text{acl}_{\mathcal{W}}$, the reader's public key $pk_{\mathcal{R}}$, the policy policy , and outputs a converted ciphertext $\bar{\text{ct}}_i$ or a failure symbol $\perp$.
- $\text{msg}'/\perp \leftarrow \text{Retrieve}(sk_{\mathcal{R}}, \{(i_j, \bar{\text{ct}}_{i_j})\}_{j \in [n']})$. The Retrieve algorithm takes as input the reader's secret key $sk_{\mathcal{R}}$ and a collection of converted ciphertexts $\{\bar{\text{ct}}_{i_j}\}_{j \in [n']}$ and outputs a message msg' or a failure symbol $\perp$.

For an ECA, we consider *token verifiability*, *t-robustness* and security against *t-collusion resistant chosen-plaintext attacks* (*t-cr-CPA security*).

Token Verifiability. It requires that the validity of individual tokens and the consistency of all the tokens under the public key of a writer can be verified. See Def. 4 for a formal description.

t-Robustness. Even if there are at most t malicious servers who may contribute fake converted ciphertexts, the reader $\mathcal{R}$ can always successfully retrieve the messages whose access control lists are consistent with the policy. See Def. 5 for a formal description.

t-cr-CPA Security. It requires the computational indistinguishability of the ciphertexts and converted ciphertexts for any two plaintexts of the adversary's choice, even if the adversary corrupts at most t servers by obtaining their tokens. We consider CPA security because it seems hard to achieve CCA security in ECA, similar to proxy re-encryption. See Def. 6 for a formal description.

Definition 4 (Token Verifiability of ECA.). Let $\text{pp} \leftarrow \text{Setup}$.

- **Validity of Individual Tokens.** If $(pk_{\mathcal{W}}, sk_{\mathcal{W}}) \leftarrow \text{KeyGen1}$ and $(\text{token}_1, \dots, \text{token}_n) \leftarrow \text{Authorize}(sk_{\mathcal{W}})$, then for all $i \in [n]$,

$$\text{Certify}(pk_{\mathcal{W}}, i, \text{token}_i) = 1.$$

- **Consistency of Tokens and Public Key.** Given $pk_{\mathcal{W}}$ and $\{\text{token}'_i\}_{i \in [n]}$, if $\text{Certify}(pk_{\mathcal{W}}, i, \text{token}'_i) = 1$ for all $i \in [n]$, then $(\text{token}_1, \dots, \text{token}_n)$ must be a correct output from KeyGen1 and Authorize , i.e., there must exist randomness r, r' such that

$$\text{KeyGen1}(r) = (pk_{\mathcal{W}}, sk_{\mathcal{W}}),$$

$$\text{Authorize}(sk_{\mathcal{W}}; r') = (\text{token}'_1, \text{token}'_2, \dots, \text{token}'_n).$$

Definition 5 (t -Robustness of ECA). Let $\text{ECA} = (\text{Setup}, \text{KeyGen1}, \text{KeyGen2}, \text{Encrypt}, \text{Authorize}, \text{Certify}, \text{Convert}, \text{Retrieve})$ as per Def. 3. ECA has t -robustness if for any (even all-powerful) $\mathcal{A}$, it holds that $|\Pr[\text{Exp}_{\text{ECA}, \mathcal{A}}^{t\text{-rob}} \Rightarrow 1]| = 1$, where $\text{Exp}_{\text{ECA}, \mathcal{A}}^{t\text{-rob}}$ is an experiment played between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$, as described as follows and also formalized in Fig. 2.

$\text{Exp}_{\text{ECA}, \mathcal{A}}^{t\text{-rob}}$:

1. Challenger $\mathcal{C}$ sets up ECA and generates the public/secret key pairs for Writer $\mathcal{W}$ and Reader $\mathcal{R}$:
 $\text{pp} \leftarrow \text{Setup}; (pk_{\mathcal{W}}, sk_{\mathcal{W}}) \leftarrow \text{KeyGen1}; (pk_{\mathcal{R}}, sk_{\mathcal{R}}) \leftarrow \text{KeyGen2}.$
Challenger $\mathcal{C}$ sends $(\text{pp}, pk_{\mathcal{W}}, pk_{\mathcal{R}})$ to adversary $\mathcal{A}$.
2. Challenger $\mathcal{C}$ generates all the tokens via $(\text{token}_1, \dots, \text{token}_n) \leftarrow \text{Authorize}(sk_{\mathcal{W}}).$
3. Adversary $\mathcal{A}$ submits to $\mathcal{C}$ message $\text{msg} \in \mathcal{M}$ and access control list $\text{acl}_{\mathcal{W}}$ such that $\text{policy}(\text{acl}_{\mathcal{W}}, pk_{\mathcal{R}}) = 1$.
4. Challenger $\mathcal{C}$ invokes $(\text{ct}, \text{acl}_{\mathcal{W}}) \leftarrow \text{Encrypt}(pk_{\mathcal{W}}, \text{msg}, \text{acl}_{\mathcal{W}})$ and $\bar{\text{ct}}_i \leftarrow \text{Convert}(\text{token}_i, (pk_{\mathcal{W}}, \text{ct}, \text{acl}_{\mathcal{W}}), pk_{\mathcal{R}}, \text{policy})$ to obtain the ciphertext ct and all the corresponding converted ciphertexts $\{\bar{\text{ct}}_i\}_{i \in [n]}$ for msg .
5. Adversary $\mathcal{A}$ submits to $\mathcal{C}$ with a collection of converted ciphertexts $\{\bar{\text{ct}}'_i\}_{i \in [n]}$, among which $\mathcal{A}$ can replace at most t correctly converted ciphertexts with bad ones (i.e., $|\{\bar{\text{ct}}'_i\}_{i \in [n]} \cap \{\bar{\text{ct}}_i\}_{i \in [n]}| \geq n - t$).
6. Challenger $\mathcal{C}$ tests whether $\text{Retrieve}(sk_{\mathcal{R}}, \{(i_j, \bar{\text{ct}}'_i)\}_{i \in [n]}) = \text{msg}$. The experiment outputs 1 if Retrieve correctly retrieves msg , and outputs 0 otherwise.

The t -cr-CPA Security of ECA is formalized in Def. 6.

Definition 6 (t -cr-CPA Security of ECA). Let $\text{ECA} = (\text{Setup}, \text{KeyGen1}, \text{KeyGen2}, \text{Encrypt}, \text{Authorize}, \text{Certify}, \text{Convert}, \text{Retrieve})$ be an ECA system. ECA is t -cr-CPA secure if for any PPT $\mathcal{A}$, it holds that $\text{Adv}_{\text{ECA}, \mathcal{A}}^{t\text{-cr-cpa}}(\kappa) := |\Pr[\text{Exp}_{\text{ECA}, \mathcal{A}}^{t\text{-cr-cpa}} \Rightarrow 1] - \frac{1}{2}| \leq \text{negl}(\kappa)$, where $\text{Exp}_{\text{ECA}, \mathcal{A}}^{t\text{-cr-cpa}}$ is an experiment played between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$, as shown as follows and also formalized in Fig. 3.

$\text{Exp}_{\text{ECA}, \mathcal{A}}^{t\text{-cr-cpa}}$:

1. Challenger $\mathcal{C}$ sets up the ECA system with $\text{pp} \leftarrow \text{Setup}$ and sends pp to adversary $\mathcal{A}$.
2. Adversary $\mathcal{A}$ issues a corruption query which consists of at most t token indices $(i_1, i_2, \dots, i_{t'})$ (with $t' \leq t$) which indicates the tokens that $\mathcal{A}$ would like to obtain.

```

ExpECA, At-rob:
pp ← Setup
(pkW, skW) ← KeyGen1
(pkR, skR) ← KeyGen2
(token1, ..., tokenn) ← Authorize(skW)
(msg, aclW, policy) ← A(pkW, pkR)
(ct, aclW) ← Encrypt(pkW, msg, aclW)
For i ∈ [n]:
    cti ← Convert(tokeni, (pkW, ct, aclW), pkR, policy)
(ct'1, ..., ct'n) ← A(st', ct, ct1, ..., ctn)
If |{ct'i}i∈[n] ∩ {cti}i∈[n]| < n - t
    Return 1
If Retrieve(skR, {(ij, ct'ij'})j∈[n']) = msg: Return 1;
Else: Return 0

```

Fig. 2. The t -robustness experiment $\text{Exp}_{\text{ECA}, \mathcal{A}}^{t\text{-rob}}$ for ECA.

3. Challenger $\mathcal{C}$ generates the public/secret key pairs for Writer $\mathcal{W}$ and Reader $\mathcal{R}$ by invoking $(pk_{\mathcal{W}}, sk_{\mathcal{W}}) \leftarrow \text{KeyGen1}$ and $(pk_{\mathcal{R}}, sk_{\mathcal{R}}) \leftarrow \text{KeyGen2}$. Then $\mathcal{C}$ generates all the tokens via $(\text{token}_1, \dots, \text{token}_n) \leftarrow \text{Authorize}(sk_{\mathcal{W}})$. Challenger $\mathcal{C}$ sends $((pk_{\mathcal{W}}, pk_{\mathcal{R}}), (\text{token}_{i_1}, \text{token}_{i_2}, \dots, \text{token}_{i_{t'}}))$ to $\mathcal{A}$.
4. Adversary $\mathcal{A}$ issues two messages $\text{msg}_0, \text{msg}_1$ of equal length, an access control list $\text{acl}_{\mathcal{W}}$ and a policy policy to $\mathcal{C}$ such that $\text{policy}(\text{acl}_{\mathcal{W}}, pk_{\mathcal{R}}) = 1$.
5. Challenger $\mathcal{C}$ randomly chooses a bit $\gamma \leftarrow_s \{0, 1\}$ and invokes

$$(\text{ct}, \text{acl}_{\mathcal{W}}) \leftarrow \text{Encrypt}(pk_{\mathcal{W}}, \text{msg}_{\gamma}, \text{acl}_{\mathcal{W}}),$$

$$\bar{\text{ct}}_i \leftarrow \text{Convert}(\text{token}_i, (pk_{\mathcal{W}}, \text{ct}, \text{acl}_{\mathcal{W}}), pk_{\mathcal{R}}, \text{policy}) \text{ for } i \in [n].$$

Then $\mathcal{C}$ sends $(\text{ct}, \bar{\text{ct}}_1, \dots, \bar{\text{ct}}_n)$ to $\mathcal{A}$.

6. $\mathcal{A}$ outputs a guessing bit γ' . If $\gamma' = \gamma$, the experiment returns 1; otherwise, it returns 0.

Given $\text{ECA} = (\text{Setup}, \text{KeyGen1}, \text{KeyGen2}, \text{Encrypt}, \text{Authorize}, \text{Certify}, \text{Convert}, \text{Retrieve})$, the threshold encryption with controlled authorization works in the following framework.

The Setup Phase. The system invokes $\text{pp} \leftarrow \text{Setup}$ to obtain and publish the system parameter pp . The policy policy is instantiated. Note that $\text{policy}(\cdot, \cdot)$ is a predicate and $\text{policy}(\text{acl}_{\mathcal{W}}, pk_{\mathcal{R}}) = 1$ means that this reader has access to the message according to the access control list $\text{acl}_{\mathcal{W}}$.

Writer's Messaging. A writer invokes $(pk_{\mathcal{W}}, sk_{\mathcal{W}}) \leftarrow \text{KeyGen1}$ to obtain its key pair. Then the writer prepares the messages msg to be written and determines the access control list $\text{acl}_{\mathcal{W}}$ associated with the message. Then it encrypts msg via the Encrypt algorithm and obtains $(\text{ct}, \text{acl}_{\mathcal{W}}) \leftarrow \text{Encrypt}(sk_{\mathcal{W}}, \text{msg}, \text{acl}_{\mathcal{W}})$. The writer sends $(\text{ct}, \text{acl}_{\mathcal{W}})$ to all the servers $\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_n$.

```

 $\text{Exp}_{\text{ECA}, \mathcal{A}}^{t\text{-cr-cpa}}:$ 
 $\text{pp} \leftarrow \text{Setup}$ 
 $(i_1, i_2, \dots, i_{t'}, \text{st}) \leftarrow \mathcal{A}(\text{pp})$ 
 $(pk_{\mathcal{W}}, sk_{\mathcal{W}}) \leftarrow \text{KeyGen1}$ 
 $(pk_{\mathcal{R}}, sk_{\mathcal{R}}) \leftarrow \text{KeyGen2}$ 
 $(\text{token}_1, \dots, \text{token}_n) \leftarrow \text{Authorize}(sk_{\mathcal{W}}).$ 
 $(\text{msg}_0, \text{msg}_1, \text{acl}_{\mathcal{W}}, \text{policy}, \text{st}') \leftarrow \mathcal{A}(\text{st}, pk_{\mathcal{W}}, pk_{\mathcal{R}}, \{\text{token}_{i_j}\}_{j \in [t']})$ 
 $\gamma \leftarrow_{\$} \{0, 1\}$ 
 $(\text{ct}, \text{acl}_{\mathcal{W}}) \leftarrow \text{Encrypt}(pk_{\mathcal{W}}, \text{msg}_{\gamma}, \text{acl}_{\mathcal{W}})$ 
For  $i \in [n]$ :
     $\bar{\text{ct}}_i \leftarrow \text{Convert}(\text{token}_i, (pk_{\mathcal{W}}, \text{ct}, \text{acl}_{\mathcal{W}}), pk_{\mathcal{R}}, \text{policy})$ 
 $\gamma' \leftarrow \mathcal{A}(\text{st}', \text{ct}, \bar{\text{ct}}_1, \dots, \bar{\text{ct}}_n)$ 
If  $\gamma' = \gamma$ : Return 1;
Else: Return 0

```

Fig. 3. The t -cr-CPA security experiment $\text{Exp}_{\text{ECA}, \mathcal{A}}^{t\text{-cr-cpa}}$ for ECA.

Write Authorization. For $i \in [n]$, the writer invokes $(\text{token}_1, \dots, \text{token}_n) \leftarrow \text{Authorize}(sk_{\mathcal{W}})$ and sends token_i to server $\mathcal{S}_i$ via a private channel. Then server $\mathcal{S}_i$ tests the validity of token_i by checking whether $\text{Certify}(pk_{\mathcal{W}}, i, \text{token}_i) = 1$. If the output of Certify is 0, server $\mathcal{S}_i$ will report the invalidity of its token to the writer and all the servers. In case of any report of invalid tokens, the writer will re-issue the correct tokens to make sure that every server has a valid token.

Matching Reader's Request. A reader invokes $(pk_{\mathcal{R}}, sk_{\mathcal{R}}) \leftarrow \text{KeyGen2}$ to obtain its own key pair. Then the reader sends its public key $pk_{\mathcal{R}}$ to all the servers $\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_n$. After receiving a ciphertext ct and its associated access control list $\text{acl}_{\mathcal{W}}$, for each reader $\mathcal{R}$ associated with $pk_{\mathcal{R}}$, each server $\mathcal{S}_i$ ($i \in [n]$) checks whether policy matches $\text{acl}_{\mathcal{W}}$. $\text{policy}(\text{acl}_{\mathcal{W}}, pk_{\mathcal{R}}) = 0$ means that $\mathcal{R}$ did not have access to the message with $\text{acl}_{\mathcal{W}}$, and hence server $\mathcal{S}_i$ does nothing for $\mathcal{R}$. Otherwise, $\text{policy}(\text{acl}_{\mathcal{W}}, pk_{\mathcal{R}}) = 1$, and then $\mathcal{S}_i$ invokes $\bar{\text{ct}}_i \leftarrow \text{Convert}(\text{token}_i, (\text{ct}, \text{acl}_{\mathcal{W}}), pk_{\mathcal{R}}, \text{policy})$ and sends $\bar{\text{ct}}_i$ to $\mathcal{R}$.

Reader's Message Retrieval. Reader $\mathcal{R}$ collects converted ciphertexts from servers. After a collection of at least t , say n' , converted ciphertexts $\{\bar{\text{ct}}_{i_j}\}_{j \in [n']}$, reader $\mathcal{R}$ invokes $\text{msg}'/\perp \leftarrow \text{Retrieve}(sk_{\mathcal{R}}, \{\bar{\text{ct}}_{i_j}\}_{j \in [n']})$ to retrieve the requested message.

By *token verifiability*, we can make sure that all the tokens are correctly generated by the writer, and hence *token verifiability* eliminates the possibility that a malicious writer cheats servers by issuing fake tokens. By *t-robustness* of ECA, we know that the reader can always recover the correctly written message msg , even if t (or less than t) malicious servers collude.

To capture the security of ECA system, we consider such a scenario: the public keys of both the writer and the reader are known by everyone, and of course by the adversary $\mathcal{A}$. Some servers may collude together, and we consider collusion of at most t servers. This can be formalized by allowing $\mathcal{A}$ to corrupt up to t servers and get their tokens. We consider security against chosen-plaintext attacks, so we allow $\mathcal{A}$ to choose two plaintext messages msg_0 and msg_1 for writing. The writer invokes **Encrypt** to encrypt msg_0 or msg_1 with equal probability and issues the corresponding ciphertext ct to $\mathcal{A}$. Then server $\mathcal{S}_i$ converts ct to $\bar{\text{ct}}_i$ for $i \in [n]$, and $\mathcal{A}$ also obtains the converted ciphertexts $\{\bar{\text{ct}}_i\}_{i \in [n]}$. The security requires that for any PPT adversary $\mathcal{A}$, it is hard for it to tell whether ct is the ciphertext and $\{\bar{\text{ct}}_i\}_{i \in [n]}$ are the converted ciphertexts of msg_0 or msg_1 . With such a security, $\mathcal{A}$ learns no information on the plaintext message from ct and $\{\bar{\text{ct}}_i\}_{i \in [n]}$, even if it also obtains at most t tokens. Such a security requirement is exactly captured by t -cr-CPA security of ECA as defined in Def. 6.

In our formalization, writers and readers are decoupled, in the sense they do not need to interact with each other. As argued in Chios [13], the feature offers both flexibility and scalability, enabling applications such as publish/subscribe messaging.

5 Construction of ECA from the Square-DDH Assumption

Let **PGGen** be a PPT bilinear group generation algorithm, which outputs a description of pairing group $\text{gpar} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, e, g_1, g_2, g_T = e(g_1, g_2))$, where $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ are multiplicative cyclic groups of prime order p , the map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is a non-degenerated bilinear pairing, and g_1, g_2 are generators of $\mathbb{G}_1, \mathbb{G}_2$, respectively. Consequently, $g_T := e(g_1, g_2)$ is a generator of $\mathbb{G}_T$. This bilinear group can be instantiated with *Type-III asymmetric pairing group*, where $\mathbb{G}_1 \neq \mathbb{G}_2$, no known efficiently computable isomorphisms from $\mathbb{G}_1$ to $\mathbb{G}_2$ or $\mathbb{G}_2$ to $\mathbb{G}_1$ exist, and the operations in $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ and the pairing e are efficiently computable.

Let $\text{SE} = (\text{SE.Enc}, \text{SE.Dec})$ be a symmetric encryption scheme with key space $\mathbb{G}_T$. This can be instantiated with the NIST standard algorithm AES.

Let $\text{NIZK}_{\text{eq}} = (\text{ZKProof}, \text{ZKVerify}, \text{ZKSim})$ be the NIZK proof system for $\mathcal{L}_{\text{ddh}}$, the language of DDH tuples (see subsec. 3.2).

In our ECA system, writers can take flexible control of the collaboration with servers. If $pk_{\mathcal{W}}$ stops issuing ciphertexts $\text{ct} \leftarrow \text{Encrypt}(pk_{\mathcal{W}}, \text{msg}, \text{acl}_{\mathcal{W}})$, the collaboration with servers terminates. Writers can also start a new collaboration with a new set of servers by generating another public/secret key pair and issuing the corresponding tokens to the new set of servers. In this way, message confidentiality is taken by writers with flexible control.

Theorem 1. *The construction of ECA has the property of token verifiability.*

Proof. Validity of Individual Tokens. Suppose that the output of **KeyGen1** results in $pk_{\mathcal{W}} = (g_1^{1/\alpha}, (v_0 = g_T^\alpha, v_1 = g_T^{r_1}, \dots, v_t = g_T^{r_t}), (w_1, \dots, w_n))$ and

$\text{pp} \leftarrow \text{Setup};$ $\text{gpar} \leftarrow \text{PGGen}.$ $// \text{gpar} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, e, g_1, g_2, g_T = e(g_1, g_2))$ $H : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ is a hash function. Return $\text{pp} := (\text{gpar}, H).$ $(pk_W, sk_W) \leftarrow \text{KeyGen1}(\text{pp}):$ $\alpha \leftarrow \mathbb{Z}_p, v_0 := g_T^\alpha.$ For $j \in [t]: r_j \leftarrow \mathbb{Z}_p, v_j := g_T^{r_j}.$ $sk_W := (\alpha, r_1, \dots, r_t).$ $r(x) := \alpha + \sum_{j=1}^t r_j \cdot x^j \mod p.$ For $i \in [n]:$ $\theta_i := r(i), w_i = e(g_1^{1/\alpha}, g_2^{\theta_i}).$ $pk_W := (g_1^{1/\alpha}, (v_0, v_1, \dots, v_t), (w_1, \dots, w_n)).$ Return $(pk_W, sk_W).$ $(pk_R, sk_R) \leftarrow \text{KeyGen2}(\text{pp}):$ $\beta \leftarrow \mathbb{Z}_p, sk_R := \beta, pk_R := g_2^\beta.$ Return $(pk_R, sk_R).$ $(ct, acl_W) \leftarrow \text{Encrypt}(pk_W, \text{msg}, acl_W):$ Parse $pk_W = (v', (v_0, v_1, \dots, v_t), (w_1, w_2, \dots, w_n)).$ $r \leftarrow \mathbb{Z}_p, ct_1 := v'^r, k := g_T^r.$ $ct_2 \leftarrow \text{SE.Enc}(k, \text{msg}).$ Return $(ct = (ct_1, ct_2), acl_W).$ $(\text{token}_1, \dots, \text{token}_n) \leftarrow \text{Authorize}(sk_W):$ Parse $sk_W = (\alpha, r_1, \dots, r_t).$ $r(x) := \alpha + \sum_{j=1}^t r_j \cdot x^j \mod p.$ For $i \in [n]:$ $\theta_i := r(i).$ $\text{token}_i := \theta_i.$ Return $(\text{token}_1, \dots, \text{token}_n).$ $0/1 \leftarrow \text{Certify}(pk_W, i, \text{token}_i):$ // validity of tokens Parse $pk_W = (v', (v_0, v_1, \dots, v_t), (w_1, w_2, \dots, w_n)).$ Parse $\text{token}_i = \theta_i.$ //test consistency between θ_i, w_i and $(v', v_0, \dots, v_t)$ If $(g_T^{\theta_i} \neq \prod_{\ell=0}^t v_\ell^{\theta_i}) \vee (e(v', g_2^{\theta_i}) \neq w_i):$ Return 0. //token invalid For $k \in [t+1], j \in [t+2, n]:$ $\lambda_{kj} := \prod_{i=1, i \neq k}^n \frac{j-i}{k-i}. // \text{Lagrange coefficients}$ //test consistency between v' and $(w_1, \dots, w_{t+1})$ If $(\prod_{k=1}^{t+1} w_k^{\lambda_{k0}} \neq g_T):$ Return 0; //token invalid //test consistency of $(w_1, \dots, w_n)$ For $j \in [t+2, n]$ If $\prod_{k=1}^{t+1} w_k^{\lambda_{kj}} \neq w_j:$ Return 0; //token invalid Else: Return 1. //token valid	$\tilde{ct}_i \leftarrow \text{Convert}(\text{token}_i, (pk_W, ct, acl_W), pk_R = g_2^\beta, \text{policy}):$ If $\text{policy}(acl_W, pk_R) = 0:$ Return $\perp.$ Parse $\text{token}_i = \theta_i.$ Parse $ct = (ct_1, ct_2).$ $h := e(ct_1, pk_R) = e(ct_1, g_2^\beta), \bar{h}_i := h^{\theta_i}.$ $// \pi_i \leftarrow \text{ZKProof}((h, \bar{h}_i, g_T, \bar{u}_i), \theta_i)$ $\bar{u}_i := g_T^{\theta_i}.$ $s_i \leftarrow \mathbb{Z}_p, \hat{h}_i := h^{s_i}, \hat{u}_i := g_T^{s_i}.$ $e_i := H(h, g_T, \bar{h}_i, \bar{u}_i, \hat{h}_i, \hat{u}_i).$ $f_i := s_i + \theta_i \cdot e_i \mod p.$ $\pi_i := (\hat{h}_i, \hat{u}_i, f_i). // \text{proof of } \log_h \bar{h}_i = \log_{g_T} \bar{u}_i$ $\tilde{ct}_{i,0} := h.$ $\tilde{ct}_{i,1} := (\hat{h}_i, \pi_i), \tilde{ct}_{i,2} := ct_2.$ Return $\tilde{ct}_i = (\tilde{ct}_{i,0}, \tilde{ct}_{i,1}, \tilde{ct}_{i,2}).$ $\text{msg}' / \perp \leftarrow \text{Retrieve}(sk_R = \beta, \{(\ell_j, \tilde{ct}_{\ell_j})\}_{j \in [n]}):$ If $n' \leq t:$ Return $\perp.$ Parse $\tilde{ct}_{\ell_j} = (\tilde{ct}_{\ell_j,0}, \tilde{ct}_{\ell_j,1}, \tilde{ct}_{\ell_j,2})$ for $j \in [n'].$ Filter set $\{(\ell_j, \tilde{ct}_{\ell_j})\}_{j \in [n']}$ to obtain subset $\mathcal{S} =$ $\{(i_j, \tilde{ct}_{i_j})\}_{j \in [n']}$ satisfying $n'' \geq t+1$ $\bigwedge_{j, j' \in [n'']} ((\tilde{ct}_{i_j,0} = \tilde{ct}_{i_{j'},0}) \wedge (\tilde{ct}_{i_j,2} = \tilde{ct}_{i_{j'},2})): \quad (\star)$ $\mathcal{V} := \emptyset.$ For $j \in [n'']$ Parse $\tilde{ct}_{i_j,0} = h, \tilde{ct}_{i_j,1} = (\hat{h}_{i_j}, \pi_{i_j} = (\hat{h}_{i_j}, \hat{u}_{i_j}, f_{i_j})).$ $// 0/1 \leftarrow \text{ZKVerify}((h, \hat{h}_{i_j}, g_T, \hat{u}_{i_j}), \pi_{i_j})$ $\bar{u}_{i_j}' := \prod_{\ell=0}^t (v_\ell)^{f_{i_j}^\ell}.$ $e_{i_j} := H(h, g_T, \bar{h}_{i_j}, \bar{u}_{i_j}', \hat{h}_{i_j}, \hat{u}_{i_j}).$ If $(h^{f_{i_j}} = \hat{h}_{i_j} \cdot \bar{h}_{i_j}^{e_{i_j}}) \wedge (g_T^{f_{i_j}} = \hat{u}_{i_j} \cdot \bar{u}_{i_j}'^{e_{i_j}}): \quad (\star\star)$ $\mathcal{V} := \mathcal{V} \cup \{(i_j, \bar{h}_{i_j})\}.$ // end of proof verification of $\log_h \bar{h}_{i_j} = \log_{g_T} \bar{u}_{i_j}'$ If $\mathcal{V} \leq t:$ Return $\perp.$ Parse $\mathcal{V} = \{(j_0, \bar{h}_{j_0}), (j_1, \bar{h}_{j_1}), \dots, (j_t, \bar{h}_{j_t}), \dots\}.$ $k' := \left(\prod_{i=0}^t \bar{h}_{j_i}^{\lambda_{j_i,0}} \right)^{\beta^{-1}}. // \lambda_{j_i,0}$'s are Lagrange coefficients $\text{msg}' \leftarrow \text{SE.Dec}(k', \tilde{ct}_{j_1,2}). // \tilde{ct}_{j_1,2} = \tilde{ct}_{j_2,2} = \dots = \tilde{ct}_{j_t,2}$ Return $\text{msg}'.$
--	---

Fig. 4. Construction of ECA from square-DDH.

$sk_{\mathcal{W}} = (\alpha, r_1, \dots, r_t)$. Let the output of $\text{Authorize}(sk_{\mathcal{W}})$ be $\{\text{token}_i = \theta_i\}_{i \in [n]}$. Let $r(x) := \alpha + r_1x + \dots + r_tx^t$. Then

$$\theta_i = r(i) = \alpha + r_1 \cdot i + r_2 \cdot i^2 + \dots, r_t \cdot i^t \pmod{p}, \quad (3)$$

$$w_i = e(g_1^{1/\alpha}, g_2^{\theta_i}). \quad (4)$$

Consequently, we have

$$g_T^{\theta_i} = g_T^{\alpha + r_1 \cdot i + r_2 \cdot i^2 + \dots, r_t \cdot i^t} = \prod_{\ell=0}^t v_{\ell}^{i^{\ell}}. \quad (5)$$

Meanwhile, given the Lagrange coefficients $\lambda_{kj} := \prod_{i=1, i \neq k}^n \frac{j-i}{k-i}$ for $k \in [n+1], j \in [0, n]$, we have

$$\begin{aligned} \prod_{k=1}^{t+1} w_k^{\lambda_{k0}} &= \prod_{k=1}^{t+1} e(g_1, g_2)^{\frac{\theta_k}{\alpha} \cdot \lambda_{k0}} = e(g_1, g_2)^{\frac{1}{\alpha} \cdot \sum_{k=1}^{t+1} (\theta_k \cdot \lambda_{k0})} = e(g_1, g_2)^{\frac{1}{\alpha} \cdot \alpha} = g_T, \quad (6) \\ \prod_{k=1}^{t+1} w_k^{\lambda_{kj}} &= \prod_{k=1}^{t+1} e(g_1^{\frac{1}{\alpha}}, g_2^{\theta_k})^{\lambda_{kj}} = e(g_1^{\frac{1}{\alpha}}, g_2^{\sum_{k=1}^{t+1} (\theta_k \cdot \lambda_{kj})}) = e(g_1^{\frac{1}{\alpha}}, g_2^{\theta_j}) = w_j, \quad j \in [n]. \end{aligned} \quad (7)$$

Therefore, $\text{Certify}(pk_{\mathcal{W}}, i, \text{token}_i) = 1$ for all $i \in [n]$.

Consistency of Tokens & Public Key. Suppose that $pk_{\mathcal{W}} = (g_1^{1/\alpha}, (v_0, v_1, \dots, v_t), (w_1, \dots, w_n))$ and $\text{token}'_i = \theta_i$ for $i \in [n]$. Then $\text{Certify}(pk_{\mathcal{W}}, i, \text{token}'_i) = 1$ implies that (4), (5), (6), (7) hold.

Define $r_j := \log_{g_T} v_j$ for $j \in [0, t]$ and $r(x) := r_0 + r_1x + \dots + r_tx^t$. By (5), we know that

$$\theta_i = r(i) = r_0 + r_1 \cdot i + r_2 \cdot i^2 + \dots, r_t \cdot i^t \pmod{p}. \quad (8)$$

By (4) and (6), we know that

$$r_0 = r(0) = \sum_{k=1}^{t+1} (\theta_k \cdot \lambda_k) = \alpha. \quad (9)$$

Therefore, $r = (r_0 = \alpha, r_1 = \log_{g_T} v_1, \dots, r_t = \log_{g_T} v_t)$ is just the randomness such that $\text{KeyGen1}(r) = (pk_{\mathcal{W}}, sk_{\mathcal{W}})$, and $sk_{\mathcal{W}} = (\alpha, r_1, \dots, r_t)$. Moreover, (8), (9), and (4) imply that $\text{Authorize}(sk_{\mathcal{W}} = \alpha) = (\text{token}'_1, \dots, \text{token}'_n)$. $\square$

Theorem 2. *The construction of ECA has the property of t -robustness if $n \geq 2t + 1$.*

Proof. Given $pk_{\mathcal{W}} = (g_1^{1/\alpha}, (v_0, v_1, \dots, v_t), (w_1, \dots, w_n))$, $sk_{\mathcal{W}} = (\alpha, r_1, \dots, r_n)$, the output of Authorize yields $\text{token}_i = \theta_i$ for $i \in [n]$, where

$$\theta_i = r(i) = \alpha + r_1 \cdot i + r_2 \cdot i^2 + \dots, r_t \cdot i^t \pmod{p},$$

$$v_0 = g_T^\alpha, v_1 = g_T^{r_1}, \dots, v_t = g_T^{r_t}.$$

If ct is the output of $\text{Encrypt}(pk_{\mathcal{W}}, \text{msg}, \text{acl}_{\mathcal{W}})$, then

$$\text{ct} = (\text{ct}_1, \text{ct}_2), \text{ with } \text{ct}_1 = g_1^{r/\alpha} \text{ and } \text{ct}_2 = \text{SE.Enc}(k = g_T^r, \text{msg}).$$

Algorithm $\text{Convert}(\text{token}_i, (pk_{\mathcal{W}}, \text{ct}, \text{acl}_{\mathcal{W}}), pk_{\mathcal{R}}, \text{policy})$ will result in converted ciphertext $\bar{\text{ct}}_i = (\bar{\text{ct}}_{i,0}, \bar{\text{ct}}_{i,1}, \bar{\text{ct}}_{i,2})$ with

$$\bar{\text{ct}}_{i,0} = h = e(g_1^{r/\alpha}, g_2^\beta) = g_T^{\beta r/\alpha}, \quad \bar{\text{ct}}_{i,1} = (\bar{h}_i, \pi_i = (\hat{h}_i, \hat{u}_i, f_i)), \quad \bar{\text{ct}}_{i,2} = \text{ct}_2,$$

where π_i is the output of $\text{ZKProof}((h, \bar{h}_i, g_T, \bar{u}_i), \theta_i)$, i.e.,

$$\begin{aligned} \bar{h}_i &= h^{\theta_i}, \quad \bar{u}_i = g_T^{\theta_i}, \quad \hat{h}_i = h^{s_i}, \quad \hat{u}_i = g_T^{s_i} \\ e_i &= H(h, g_T, \bar{h}_i, \bar{u}_i, \hat{h}_i, \hat{u}_i), \quad f_i = s_i + \theta_i \cdot e_i \pmod{p}. \end{aligned} \quad (10)$$

Due to the fact that $f_i = s_i + \theta_i \cdot e_i \pmod{p}$ for $i \in [t]$, it must hold that

$$(h^{f_i} = h^{s_i + \theta_i \cdot e_i} = \hat{h}_i \bar{h}_i^{e_i}) \text{ and } (g_T^{f_i} = g_T^{s_i + \theta_i \cdot e_i} = \hat{u}_i \bar{u}_i^{e_i}).$$

Let set $\mathcal{S}' := \{\bar{\text{ct}}'_i\}_{i \in [n]}$ be the converted ciphertexts submitted to Retrieve by the adversary. Now let us consider the execution of $\text{Retrieve}(sk_{\mathcal{R}} = \beta, \mathcal{S}' = \{\bar{\text{ct}}'_i\}_{i \in [n]})$.

First, Retrieve will find the subset $\mathcal{S} = \{\bar{\text{ct}}'_{i_j}\}_{j \in [n'']} (\subseteq \mathcal{S}')$ satisfying $n'' \geq t+1$ and $(\star)$. Recall that there are at most t fake converted ciphertexts, so $\mathcal{S}$ contains at least one correct converted ciphertext which is the output of Convert . The condition $(\star)$ guarantees that for all $\bar{\text{ct}}'_{i_j}$ in $\mathcal{S}$, $\bar{\text{ct}}_{i_1,0} = \dots, \bar{\text{ct}}_{i_{n''},0} = h$ are correct ones resulting from Convert , and $\bar{\text{ct}}_{i_1,2} = \dots, \bar{\text{ct}}_{i_{n''},2} = \text{ct}_2$ are the correct payload ciphertext.

Since $n \geq 2t+1$, there are at least $n-t \geq t+1$ correct converted ciphertexts (resulting from the output of Convert) and all of them must be contained in $\mathcal{S}$.

For any $\bar{\text{ct}}'_{i_j} \in \mathcal{S}$, parse $\bar{\text{ct}}'_{i_j} = (\bar{\text{ct}}'_{i_j,0}, \bar{\text{ct}}'_{i_j,1}, \bar{\text{ct}}'_{i_j,2})$ with

$$\bar{\text{ct}}'_{i_j,0} = h, \quad \bar{\text{ct}}'_{i_j,1} = (\bar{h}'_{i_j}, \pi'_{i_j} = (\hat{h}'_{i_j}, \hat{u}'_{i_j}, f'_{i_j})), \quad \bar{\text{ct}}'_{i_j,2} = \text{ct}_2.$$

Then algorithm Retrieve will compute

$$\bar{u}'_{i_j} := \prod_{\ell=0}^{t-1} (v_\ell)^{i_j^\ell} = \prod_{\ell=0}^{t-1} g_T^{r_\ell \cdot i_j^\ell} = g_T^{r(i_j)} = g_T^{\theta_{i_j}}, \quad e'_{i_j} := H(h, g_T, \bar{h}'_{i_j}, \bar{u}'_{i_j}, \hat{h}'_{i_j}, \hat{u}'_{i_j})$$

and collects $\bar{h}'_{i_j}$ from the ciphertext that satisfies

$$\hat{h}'_{i_j} \cdot (\bar{h}'_{i_j})^{e'_{i_j}} = h^{f'_{i_j}} \text{ and } \hat{u}'_{i_j} \cdot (\bar{u}'_{i_j})^{e'_{i_j}} = g_T^{f'_{i_j}}. \quad (11)$$

According to the soundness of NIZK_{eq} (see subset. 3.2), we have $\log_{g_T} \bar{u}'_{i_j} = \log_h \bar{h}'_{i_j}$ except with negligible probability. Now that $\bar{u}'_{i_j} = g_T^{\theta_{i_j}}$, so we have

$$\bar{h}'_{i_j} = h^{\theta_{i_j}}$$

for all $\bar{\text{ct}}'_{i_j} \in \mathcal{S}$ that satisfy (11).

The above analysis shows that the converted ciphertext $\bar{\text{ct}}'_i$ in $\mathcal{S}$ is correct if and only if (11) holds. Consequently, via (11) **Retrieve** can always filter $\mathcal{S}$ to obtain all the correct converted ciphertexts.

Recall that there are at most t fake converted ciphertext, so algorithm **Retrieve** can collect at least $n - t \geq t + 1$ good $\bar{h}'_{i_j}$ s (with $\bar{h}'_{i_j} = h^{\theta_{i_j}}$).

Consequently, $k' = \left(\prod_{i=1}^t \bar{h}_{j_i}^{\lambda_{j_i 0}} \right)^{\beta^{-1}} = \left(\prod_{i=1}^t h^{\theta_{j_i} \lambda_{j_i 0}} \right)^{\beta^{-1}} = h^{\beta^{-1} \sum_{i=1}^t \theta_{j_i} \lambda_{j_i 0}} = h^{\beta^{-1} \alpha} = g_T^{(\beta r \alpha^{-1})(\beta^{-1} \alpha)} = g_T^r = k$.

Now the correct symmetric key k is recovered. Together with the fact $\bar{\text{ct}}_{j_1, 2} = \text{ct}_2$, the correctness of **SE** guarantees $\text{SE.Dec}(k', \bar{\text{ct}}_{j_1, 2}) = \text{SE.Dec}(k, \text{ct}_2) = \text{msg}$. Therefore, the original message **msg** written by $\mathcal{W}$ can always be successfully retrieved even if t malicious servers collude. $\square$

Theorem 3. *The construction of ECA has t -cr-CPA security if $n \geq 2t + 1$ and the square DDH assumption holds over $\mathbb{G}_1$.*

Proof. We prove the t -cr-CPA security of ECA with a security reduction. We show that if there exists a PPT algorithm $\mathcal{A}$ winning t -cr-CPA security game with advantage $\epsilon_{\mathcal{A}}$, we can construct another PPT algorithm $\mathcal{A}'$ which can solve the square DDH problem with almost the same advantage $\epsilon_{\mathcal{A}'} \approx \epsilon_{\mathcal{A}}$.

Let $(y, y^\alpha, y^{\alpha^2}, y^r, z) \in \mathbb{G}_1^5$ be the square DDH challenge for $\mathcal{A}'$, where either $z = y^{\alpha r}$ or $z = y^c$ and $\alpha, r, c \leftarrow \mathbb{Z}_p$. The aim of $\mathcal{A}'$ is to tell whether $z = y^{\alpha r}$ or $z = y^c$.

The reduction algorithm $\mathcal{A}'$ will simulate the environment of $\text{Exp}_{\text{ECA}, \mathcal{A}}^{t\text{-cr-cpa}}$ for $\mathcal{A}$ as follows.

1. $\mathcal{A}'$ sets $g_1 := y^\alpha$. Then

$$y = g_1^{1/\alpha}, \quad y^\alpha = g_1, \quad y^{\alpha^2} = g_1^\alpha, \quad y^r = g_1^{r/\alpha}, \quad y^{\alpha r} = g_1^r.$$

$\mathcal{A}'$ chooses $g_2 \leftarrow \mathbb{G}_2$ and computes $g_T = e(g_1, g_2)$. $\mathcal{A}'$ sends $\text{pp} := ((\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, e, g_1 = y^\alpha, g_2, g_T), H)$ to $\mathcal{A}$.

2. When $\mathcal{A}$ issues a corruption query $(i_1, i_2, \dots, i_t)$ (w.l.o.g. we assume $t' = t$), $\mathcal{A}'$ randomly chooses $\theta_{i_j} \leftarrow \mathbb{Z}_p$ for $j \in [t]$ and computes

$$v_0 := e(y^{\alpha^2}, g_2) = e(g_1^\alpha, g_2) = g_T^\alpha, \quad g_T^{\theta_{i_1}}, \quad \dots, \quad g_T^{\theta_{i_t}}.$$

According to statements (1)' and (2)' of Lemma 1, Lagrange Interpolation with $(0, g_T^\alpha), (i_1, g_T^{\theta_{i_1}}), \dots, (i_t, g_T^{\theta_{i_t}})$ can yield

$$(v_1, v_2, \dots, v_t) \in \mathbb{G}_T^t \quad \text{s.t.} \quad \theta_i = \alpha + (\log_{g_T} v_1) \cdot i + (\log_{g_T} v_2) \cdot i^2 + \dots + (\log_{g_T} v_t) \cdot i^t$$

for $i \in [n]$ and yield

$$(g_T^{\theta_1}, \quad g_T^{\theta_2}, \quad \dots, \quad g_T^{\theta_n}).$$

Next, $\mathcal{A}'$ computes

$$w_{i_1} := e(g_1^{1/\alpha}, g_2^{\theta_{i_1}}), \dots, \quad w_{i_t} := e(g_1^{1/\alpha}, g_2^{\theta_{i_t}}).$$

According to the statement (2)' of Lemma 1, Lagrange Interpolation with $(0, g_T), (i_1, w_{i_1}), \dots, (i_t, w_{i_t})$ can yield

$$w_1 = e(g_1^{1/\alpha}, g_2^{\theta_1}), \quad w_2 = e(g_1^{1/\alpha}, g_2^{\theta_2}), \quad \dots, \quad w_t = e(g_1^{1/\alpha}, g_2^{\theta_t}).$$

Then $\mathcal{A}'$ sets $pk_{\mathcal{W}} := (y = g^{1/\alpha}, (v_0, v_1, \dots, v_t), (w_1, \dots, w_n))$. Note that $sk_{\mathcal{W}}$ is implicitly defined as $(\alpha, r_1 = \log_{g_T} v_1, \dots, r_t = \log_{g_T} v_t)$. $\mathcal{A}'$ also chooses $sk_{\mathcal{R}} := \beta \leftarrow_{\$} \mathbb{Z}_p$ and computes $pk_{\mathcal{R}} := g_1^\beta$. Then $\mathcal{A}'$ sends $(pp, pk_{\mathcal{W}}, pk_{\mathcal{R}})$ and $(token_{i_1} := \theta_{i_1}, \dots, token_{i_t} := \theta_{i_t})$ to adversary $\mathcal{A}$.

According to the statement (3) of Lemma 1, $\mathcal{A}'$'s simulation is perfect in this step.

3. Adversary $\mathcal{A}$ issues two messages msg_0, msg_1 of equal length, an access control list $acl_{\mathcal{W}}$ and a policy $policy$ to $\mathcal{A}'$.
4. $\mathcal{A}'$ randomly chooses a bit $\gamma \leftarrow_{\$} \{0, 1\}$, and simulates the following two algorithms

$$(ct, acl_{\mathcal{W}}) \leftarrow \text{Encrypt}(pk, msg_{\gamma}, acl_{\mathcal{W}})$$

and

$$\bar{ct}_i \leftarrow \text{Convert}(token_i, (pk_{\mathcal{W}}, ct, acl_{\mathcal{W}}), pk_{\mathcal{R}}, policy) \text{ for } i \in [n]$$

as follows. $\mathcal{A}'$ sets

$$k := z, \quad ct_1 := y^r = g_1^{r/\alpha}, \quad ct_2 := \text{SE.Enc}(k = z, msg_{\gamma}), \quad ct := (ct_1, ct_2).$$

Then it computes

$$h := e(ct_1, pk_{\mathcal{R}}) = g_T^{r\beta/\alpha}, \quad \bar{h}_{i_1} := h^{\theta_{i_1}}, \quad \dots, \quad \bar{h}_{i_t} := h^{\theta_{i_t}}.$$

$\mathcal{A}'$ applies Lagrange Interpolation with

$$(0, e(z, g_2^\beta)), \quad (i_1, \bar{h}_{i_1}), \quad \dots, \quad (i_t, \bar{h}_{i_t}) \tag{12}$$

and obtains

$$\bar{h}_1 := h^{\bar{\theta}_1}, \quad \bar{h}_2 := h^{\bar{\theta}_2}, \quad \dots, \quad \bar{h}_t := h^{\bar{\theta}_t}. \tag{13}$$

Next, $\mathcal{A}'$ invokes the simulator ZKSim of NIZK_{eq} (see subset. 3.2) to generate the simulated proof $\tilde{\pi}_i$ for the instance $(h, \bar{h}_i = h^{\bar{\theta}_i}, g_T, g_T^{\bar{\theta}_i})$ and sets

$$\bar{ct}_{i,0} := h, \quad \bar{ct}_{i,1} := (\bar{h}_i, \tilde{\pi}_i), \quad \bar{ct}_{i,2} := ct_2, \quad \bar{ct}_i = (\bar{ct}_{i,0}, \bar{ct}_{i,1}, \bar{ct}_{i,2}).$$

$\mathcal{A}'$ sends $ct = (ct_1, ct_2)$ and $\{\bar{ct}_i\}_{i \in [n]}$ to $\mathcal{A}$.

5. Finally, $\mathcal{A}$ outputs a guessing bit γ' . If $\gamma' = \gamma$, $\mathcal{A}'$ returns 1; otherwise, $\mathcal{A}'$ returns 0.

For $\mathcal{A}'$'s challenge $(y, y^\alpha, y^{\alpha^2}, y^r, z)$, we consider two cases.

- If $z = y^{\alpha r}$, then in (12) we have

$$e(z, g_2^\beta) = e(y^{\alpha r}, g_2^\beta) = e(g_1^r, g_2^\beta) = g_T^{r\beta} = h^\alpha.$$

Consequently, $\bar{\theta}_i$ in (13) satisfies $\bar{\theta}_i = \theta_i$ and the instance $(h, \bar{h}_i = h^{\theta_i}, g_T, g_T^{\theta_i})$ is indeed a DDH tuple. By the zero-knowledge property of NIZK_{eq} , the simulated $\tilde{\pi}_i$ has the identical distribution as the real proof π_i from the proving algorithm of NIZK_{eq} . Therefore, in this case, $\mathcal{A}'$ perfectly simulates $\text{Exp}_{\text{ECA}, \mathcal{A}}^{t\text{-cr-cpa}}$ for $\mathcal{A}$, and

$$\begin{aligned} & \Pr[\mathcal{A}'(y, y^\alpha, y^r, z) \Rightarrow 1 \mid \alpha, r \leftarrow_s \mathbb{Z}_p, z = y^{\alpha r}] \\ &= \Pr[\gamma = \gamma' \mid \alpha, r \leftarrow_s \mathbb{Z}_p, z = y^{\alpha r}] \\ &= \Pr[\text{Exp}_{\text{ECA}, \mathcal{A}}^{t\text{-cr-cpa}} \Rightarrow 1]. \end{aligned} \quad (14)$$

- If $z = y^c$ with $c \leftarrow_s \mathbb{Z}_p$, then $z = k$ is a random element and the CPA-security of SE makes sure that

$$\begin{aligned} & \Pr[\mathcal{A}'(y, y^\alpha, y^r, z) \Rightarrow 1 \mid \alpha, r, c \leftarrow_s \mathbb{Z}_p, z = y^c] \\ &= \Pr[\gamma = \gamma' \mid \alpha, r, c \leftarrow_s \mathbb{Z}_p, z = y^c] \in [1/2 - \text{negl}(\kappa), 1/2 + \text{negl}(\kappa)]. \end{aligned} \quad (15)$$

Subtracting (15) from (14) yields

$$\epsilon_{\mathcal{A}'} \geq \Pr[\text{Exp}_{\text{ECA}, \mathcal{A}}^{t\text{-cr-cpa}} \Rightarrow 1] - 1/2 - \text{negl}(\kappa).$$

Therefore, $\epsilon_{\mathcal{A}} = |\Pr[\text{Exp}_{\text{ECA}, \mathcal{A}}^{t\text{-cr-cpa}} \Rightarrow 1] - 1/2| \leq \epsilon_{\mathcal{A}'} + \text{negl}(\kappa)$. Given the hardness of the square DDH problem, we know that $\epsilon_{\mathcal{A}'} = \text{negl}(\kappa)$. Therefore, $\epsilon_{\mathcal{A}}$ must be $\text{negl}(\kappa)$ for any PPT adversary $\mathcal{A}$. $\square$

Remarks. Note that the threshold for our ECA scheme is $n/2$. However, it is well known that permissioned blockchains (more precisely, BFT protocols underlying permissioned blockchains) require $n \geq 3t + 1$. Therefore, if using our ECA scheme for BFT-based permissioned blockchains, we simply choose a threshold of $n/3$.

Also, while our framework is presented via a role-based access control list, our framework can equally work in the attribute-based context. Namely, the access control list can be simply replaced using attributes, and now blockchain servers match the access control policy using attributes.

In our current description, we use a trusted dealer to provide the setup parameters. However, one can use distributed key generation protocols to generate these public parameters and individual keys in a fully decentralized manner. Such protocols can work either in synchrony [26, 18, 20], partial synchrony [21, 29], or pure asynchrony [23, 12, 32].

Acknowledgement. This work was supported by the National Key R&D Program of China under Grant 2022YFB2701500. The work was also supported in part by the Major Program of Shandong Provincial Natural Science Foundation for the Fundamental Research under ZR2022ZD03.

References

- [1] Androulaki, E., Barger, A., Bortnikov, V., Cachin, C., Christidis, K., De Caro, A., Enyeart, D., Ferris, C., Laventman, G., Manevich, Y., et al.: Hyperledger fabric: a distributed operating system for permissioned blockchains. In: EuroSys '18: Proceedings of the 13th EuroSys conference. pp. 1–15 (2018)
- [2] Baek, J., Zheng, Y.: Simple and efficient threshold cryptosystem from the gap diffie-hellman group. In: GLOBECOM'03. IEEE Global Telecommunications Conference. pp. 1491–1495. IEEE (2003)
- [3] Ben-Or, M., Canetti, R., Goldreich, O.: Asynchronous secure computation. In: Proceedings of the 25th annual ACM symposium on Theory of computing. pp. 52–61 (1993)
- [4] Ben-Or, M., Kelmer, B., Rabin, T.: Asynchronous secure computations with optimal resilience. In: Proceedings of the 30th annual ACM symposium on Principles of distributed computing. pp. 183–192 (1994)
- [5] Blaze, M., Bleumer, G., Strauss, M.: Divertible protocols and atomic proxy cryptography. In: Advances in Cryptology – EUROCRYPT 1998. pp. 127–144. Springer (1998)
- [6] Boneh, D., Boyen, X., Halevi, S.: Chosen ciphertext secure public key threshold encryption without random oracles. In: Cryptographers' Track at the RSA Conference. pp. 226–243. Springer (2006)
- [7] Cachin, C., Kursawe, K., Petzold, F., Shoup, V.: Secure and efficient asynchronous broadcast protocols. In: Advances in Cryptology – CRYPTO 2001. pp. 524–541. Springer (2001)
- [8] Canetti, R., Rabin, T.: Fast asynchronous byzantine agreement with optimal resilience. In: Proceedings of the 25th annual ACM symposium on Theory of computing. pp. 42–51 (1993)
- [9] Chaum, D., Pedersen, T.P.: Wallet databases with observers. In: Advances in Cryptology – CRYPTO 1992. pp. 89–105. Springer (1992)
- [10] Cheon, J.H.: Security analysis of the strong diffie-hellman problem. In: Advances in Cryptology – EUROCRYPT 2006. pp. 1–11. Springer (2006)
- [11] Chor, B., Goldwasser, S., Micali, S., Awerbuch, B.: Verifiable secret sharing and achieving simultaneity in the presence of faults. In: Proceedings of the 26th Annual Symposium on Foundations of Computer Science (SFCS). p. 383–395. IEEE (1985)
- [12] Das, S., Xiang, Z., Kokoris-Kogias, L., Ren, L.: Practical asynchronous high-threshold distributed key generation and distributed polynomial sampling. In: 32nd USENIX Security Symposium. pp. 5359–5376 (2023)
- [13] Duan, S., Liu, C., Wang, X., Wu, Y., Xu, S., Yesha, Y., Zhang, H.: Intrusion-tolerant and confidentiality-preserving publish/subscribe messaging. In: 2020 International Symposium on Reliable Distributed Systems (SRDS). pp. 319–328. IEEE (2020)
- [14] Duan, S., Reiter, M.K., Zhang, H.: Secure causal atomic broadcast, revisited. In: 2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). pp. 61–72. IEEE (2017)
- [15] Duan, S., Wang, X., Zhang, H.: Fin: Practical signature-free asynchronous common subset in constant time. Cryptology ePrint Archive, Paper 2023/154 (2023)
- [16] Duan, S., Zhang, H.: Practical state machine replication with confidentiality. In: 2016 IEEE 35th Symposium on Reliable Distributed Systems (SRDS). pp. 187–196. IEEE (2016)

- [17] Feldman, P.: A practical scheme for non-interactive verifiable secret sharing. In: Proceedings of the 28th Annual Symposium on Foundations of Computer Science (SFCS 1987). pp. 427–438. IEEE (1987)
- [18] Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Secure distributed key generation for discrete-log based cryptosystems. In: Advances in Cryptology – EUROCRYPT 1999. pp. 295–310. Springer (1999)
- [19] Gjøsteen, K., Jager, T.: Practical and tightly-secure digital signatures and authenticated key exchange. In: Advances in Cryptology – CRYPTO 2018. pp. 95–125. Springer (2018)
- [20] Gurkan, K., Jovanovic, P., Maller, M., Meiklejohn, S., Stern, G., Tomescu, A.: Aggregatable distributed key generation. In: Advances in Cryptology – EUROCRYPT 2021. pp. 147–176. Springer (2021)
- [21] Kate, A., Huang, Y., Goldberg, I.: Distributed key generation in the wild. Cryptology ePrint Archive, Paper 2012/377 (2012)
- [22] Kokoris Kogias, E., Alp, E.C., Gasser, L., Jovanovic, P.S., Syta, E., Ford, B.A.: Calypso: Private data management for decentralized ledgers. Proceedings of the VLDB Endowment **14**(CONF), 586–599 (2021)
- [23] Kokoris Kogias, E., Malkhi, D., Spiegelman, A.: Asynchronous distributed key generation for computationally-secure randomness, consensus, and threshold signatures. In: CCS ’20: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security. p. 1751–1767 (2020)
- [24] Lu, D., Yurek, T., Kulshreshtha, S., Govind, R., Kate, A., Miller, A.: Honeybadgermpc and asynchromix: Practical asynchronous mpc and its application to anonymous communication. In: CCS ’19: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. p. 887–903 (2019)
- [25] Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system. Decentralized business review (2008)
- [26] Pedersen, T.P.: A threshold cryptosystem without a trusted party. In: Advances in Cryptology – EUROCRYPT 1991. pp. 522–526. Springer (1991)
- [27] Reiter, M.K., Birman, K.P.: How to securely replicate services. ACM Transactions on Programming Languages and Systems (TOPLAS) **16**(3), 986–1009 (1994)
- [28] Shoup, V., Gennaro, R.: Securing threshold cryptosystems against chosen ciphertext attack. In: Advances in Cryptology – EUROCRYPT 1998. pp. 1–16. Springer (1998)
- [29] Tomescu, A., Chen, R., Zheng, Y., Abraham, I., Pinkas, B., Gueta, G.G., Devadas, S.: Towards scalable threshold cryptosystems. In: 2020 IEEE Symposium on Security and Privacy (S&P). pp. 877–893. IEEE (2020)
- [30] Wood, G., et al.: Ethereum: A secure decentralised generalised transaction ledger. Ethereum project yellow paper **151**(2014), 1–32 (2014)
- [31] Yin, J., Martin, J.P., Venkataramani, A., Alvisi, L., Dahlin, M.: Separating agreement from execution for byzantine fault tolerant services. In: Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP). pp. 253–267 (2003)
- [32] Zhang, H., Duan, S., Liu, C., Zhao, B., Meng, X., Liu, S., Yu, Y., Zhang, F., Zhu, L.: Practical asynchronous distributed key generation: improved efficiency, weaker assumption, and standard model. In: 2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). pp. 568–581. IEEE (2023)
- [33] Zhou, Y., Liu, S., Han, S., Zhang, H.: Fine-grained proxy re-encryption: Definitions & constructions from lwe. Cryptology ePrint Archive, Paper 2023/1324 (2023)